

Demand Forecasting ML Development Lifecycle

Document Version: 3.0.0 | Effective Date: August 2026 | Owner: Data Science Team | Classification: Internal

Table of Contents

Introduction	2
SOP-01: Business Requirements & Use Case Definition	3
SOP-02: Data Collection & Engineering	4
SOP-03: Feature Engineering	5
SOP-04: Model Development	6
SOP-05: Model Evaluation & Validation	7
SOP-06: Deployment & Integration	8
SOP-07: Monitoring & Continuous Improvement	9
SOP-08: Governance & Compliance	10
SOP-09: Tools & Technology Standards	11
Tool Standards	11
SOP-10: Streaming Data Ingestion & Real-Time Processing	12
SOP-11: Production Monitoring & Observability	13
Appendix A: Forecast Accuracy Metrics Quick Reference	14
Appendix B: CNN-BiLSTM Architecture Reference	15

Introduction

This Standard Operating Procedure (SOP) defines the end-to-end lifecycle for developing, deploying, and maintaining machine learning models for demand forecasting. It applies to all forecasting use cases including product demand, order volumes, energy consumption, and water/utility demand prediction. The SOP is aligned with the CRISP-DM methodology, extended with MLOps practices for production-grade ML systems.

All data scientists, ML engineers, data engineers, and DevOps personnel involved in demand forecasting projects must follow this SOP. Deviations require documented approval from the Data Science Team Lead.

SOP ID	Phase	Owner	Duration (Typical)
SOP-01	Business Requirements & Use Case Definition	Product Manager + Data Science Lead	1-2 weeks
SOP-02	Data Collection & Engineering	Data Engineer	2-4 weeks
SOP-03	Feature Engineering	Data Scientist	1-3 weeks
SOP-04	Model Development	Data Scientist + ML Engineer	2-6 weeks
SOP-05	Model Evaluation & Validation	Data Scientist	1-2 weeks
SOP-06	Deployment & Integration	ML Engineer + DevOps	1-3 weeks
SOP-07	Monitoring & Continuous Improvement	ML Engineer + Data Scientist	Ongoing
SOP-08	Governance & Compliance	Data Science Lead	Ongoing
SOP-09	Tools & Technology Standards	Engineering Lead	Annual review

Table 1: SOP document structure overview

SOP-01: Business Requirements & Use Case Definition

Objective: Align ML development with business objectives before any code is written.

Procedure:

1. Identify the business problem: inventory optimization, capacity planning, procurement, or workforce scheduling. Document the specific pain point in measurable terms (e.g., "current stockout rate is 8.2%, target is <3%").
2. Define forecast granularity. Determine the level at which predictions are needed: SKU-level, product-category, regional, or national. Note that finer granularity requires more data and model complexity.
3. Determine forecast horizon. Classify as Short-term (1-7 days for operations), Medium-term (7-30 days for tactical planning), or Long-term (30-365 days for strategic decisions). Different horizons may require different models.
4. Establish success KPIs. Set primary metric (wMAPE recommended for demand forecasting), acceptable threshold (typically < 15% wMAPE), bias tolerance (within +/-5%), and business KPIs (inventory reduction %, service level %).
5. Identify stakeholders and their needs: Planners need daily forecasts with drill-down capability; Supply Chain Managers need weekly aggregated views with confidence intervals; Finance needs monthly revenue-impact projections; Executives need KPI dashboards.
6. Document data sources and availability. Create a Data Source Inventory listing each system, data type, historical range available, update frequency, and access method (API, database, file export).
7. Define acceptable latency for predictions. Real-time scoring (< 500ms per prediction) for operational use cases; batch scoring (< 30 minutes for full dataset) for planning use cases.
8. Create a Business Requirements Document (BRD) containing all above artifacts. Obtain sign-off from all stakeholders before proceeding to SOP-02.
9. Signed BRD with stakeholder approvals, KPI baseline measurements, Data Source Inventory, and Stakeholder Map (RACI matrix).

Output: Signed BRD with stakeholder approvals, KPI baseline measurements, Data Source Inventory, and Stakeholder Map (RACI matrix).

SOP-02: Data Collection & Engineering

Objective: Build a reliable, versioned data pipeline that feeds all downstream forecasting models.

Procedure:

1. Inventory data sources from the BRD. Categorize as: (a) Historical demand/sales — minimum 2-3 years for seasonal pattern capture; (b) Product metadata — category, lifecycle stage, unit price, unit cost, lead time; (c) External data — weather (temperature, precipitation, HDD/CDD), holidays, economic indicators, competitor pricing indices; (d) Real-time streaming sources — IoT meters, POS systems, ERP transaction logs, SCADA readings.
2. Assess data quality for each source. Completeness: require >95% for model training features; flag sources below threshold. Outlier detection: apply IQR method (1.5x IQR beyond Q1/Q3) and z-score method ($|z| > 3$). Missing value analysis: classify as MCAR (Missing Completely At Random), MAR (Missing At Random), or MNAR (Missing Not At Random). Document pattern and planned imputation strategy.
3. Construct the ETL/ELT pipeline using Apache Airflow or Prefect. Each pipeline must include: extraction task, validation task (Great Expectations suite), transformation task, loading task, and notification task (Slack/email on failure).
4. Implement data versioning. Use DVC for dataset versioning with remote storage (S3/MinIO). Every dataset used for model training must have a unique version tag. Alternatively, use Delta Lake for ACID-compliant versioned data lakes.
5. Perform the train/validation/test split. Use time-based splitting (NEVER random for time series). Standard ratio: 60% train, 20% validation, 20% test. For backtesting, use rolling-origin with expanding or rolling windows of configurable size.
6. Generate automated Data Quality Report covering: completeness percentages per feature, outlier counts and bounds, missing value patterns, timestamp consistency check, and distribution comparison (train vs. validation vs. test).
7. Data Quality Report (PDF/HTML), versioned datasets in DVC/Delta Lake with metadata, Airflow/Prefect pipeline DAG definition, and Great Expectations validation suite.

Output: Data Quality Report (PDF/HTML), versioned datasets in DVC/Delta Lake with metadata, Airflow/Prefect pipeline DAG definition, and Great Expectations validation suite.

SOP-03: Feature Engineering

Objective: Transform raw data into predictive features that capture temporal patterns, external influences, and domain-specific signals.

Procedure:

1. Create temporal features: (a) Lag features: $t-1$, $t-2$, $t-3$, $t-7$, $t-14$, $t-30$, $t-90$, $t-365$; (b) Rolling statistics over configurable windows: 7-day mean/std/min/max, 30-day mean/std, 90-day trend slope; (c) Calendar features: day of week (0-6), month (1-12), quarter, `is_weekend`, `is_month_start`, `is_month_end`; (d) Cyclical encodings using sin/cos transformations: `day_of_week_sin`, `day_of_week_cos`, `month_sin`, `month_cos`.
2. Create weather/climate features: Heating Degree Days ($HDD = \max(0, 18^{\circ}\text{C} - \text{avg_temp})$), Cooling Degree Days ($CDD = \max(0, \text{avg_temp} - 18^{\circ}\text{C})$), daily precipitation, cumulative 7-day and 30-day precipitation, temperature range ($\text{max} - \text{min}$), and extreme event flags (heatwave: $\text{temp} > 95\text{th percentile}$ for 3+ consecutive days; cold snap: $\text{temp} < 5\text{th percentile}$ for 3+ days).
3. Create domain features: price and promotion indicators (discount %, promotion type, days since last promotion), product lifecycle stage (ramp-up, maturity, decline — derived from launch date), inventory level flags (stockout indicator, days of supply remaining), and competitor activity indices where available.
4. Create cluster features for demand pattern segmentation: ADI (Average Demand Interval) for intermittency classification, CV-squared (coefficient of variation squared) for dispersion, seasonality strength index (ratio of seasonal variance to total variance), and revenue tier (A/B/C based on cumulative revenue contribution).
5. Validate feature quality: (a) Compute SHAP values and permutation feature importance; (b) Build correlation matrix and flag any feature pair with $|r| > 0.95$ for removal; (c) Calculate Population Stability Index (PSI) between train and test sets — flag features with $\text{PSI} > 0.25$ for investigation.
6. Feature metadata document listing all features with descriptions, Feature Importance Report (SHAP summary plot + top-20 table), Feature Correlation Matrix heatmap, and updated Feature Store (Feast) with all computed features registered.

Output: Feature metadata document listing all features with descriptions, Feature Importance Report (SHAP summary plot + top-20 table), Feature Correlation Matrix heatmap, and updated Feature Store (Feast) with all computed features registered.

SOP-04: Model Development

Objective: Train, tune, and select the best-performing model(s) using a systematic, reproducible approach.

Procedure:

1. Establish baselines first (MANDATORY — never skip). Implement and evaluate: (a) Naive forecast: use last observed value; (b) Seasonal naive: use value from same period last season; (c) Simple exponential smoothing; (d) Historical average by day-of-week. Document baseline metrics as the minimum bar for any ML model.
2. Train statistical models: SARIMA/SARIMAX with exogenous regressors using `auto_arma` for order selection; ETS (Error-Trend-Seasonality) with additive or multiplicative components; Croston's method for intermittent demand items ($ADI > 1.32$).
3. Train machine learning models: LightGBM and XGBoost gradient boosting with the full feature set. CRITICAL: ML models do not capture trend inherently — you must include trend features (cumulative time index, rolling trend slope). Use time-based cross-validation with expanding windows.
4. Train deep learning models (CNN-LSTM): Implement the reference architecture (Conv1D 64/128 filters -> MaxPool -> BiLSTM 128/64 units -> Dense). Use Adam optimizer ($lr=0.001$), `ReduceLROnPlateau` (factor=0.5, patience=10), `EarlyStopping` (patience=20). Regularization: Dropout(0.3) between CNN and LSTM layers, L2 regularization on Dense layers.
5. Build ensemble: Combine models using (a) simple average of all model predictions; (b) weighted average with weights inversely proportional to validation MAPE; (c) stacking with a meta-learner (Linear Regression or LightGBM) trained on out-of-fold predictions; (d) horizon-aware weights where different models get different weights depending on forecast step.
6. Tune hyperparameters using Optuna (Bayesian optimization) or Hyperopt. Define search spaces for each model. Use `TimeSeriesSplit` for cross-validation. Track ALL experiments in MLflow with: parameters, metrics, artifacts (model files, feature importance plots), tags (model type, data version), and git commit hash.
7. All trained models registered in MLflow Model Registry, Experiment Comparison Report showing metrics vs. baseline, hyperparameter importance analysis, and Model Card for the selected model(s).

Output: All trained models registered in MLflow Model Registry, Experiment Comparison Report showing metrics vs. baseline, hyperparameter importance analysis, and Model Card for the selected model(s).

SOP-05: Model Evaluation & Validation

Objective: Rigorously evaluate model performance using statistical metrics and business impact analysis before deployment approval.

Procedure:

1. Compute point forecast metrics on the held-out test set: MAE (unit-level error magnitude), RMSE (penalizes large errors), MAPE (percentage interpretability), sMAPE (symmetric, bounded 0-200%), wMAPE (revenue/cost-weighted — PRIMARY METRIC), MASE (scale-independent, compare to naive baseline).
2. Perform bias analysis: (a) Mean Percentage Error (MPE) — should be near zero; deviations indicate systematic over/under-forecasting; (b) Over-forecast vs. under-forecast ratio — significant asymmetry indicates model bias; (c) Bias decomposition by product category, season, volume tier, and region — identify segments where model underperforms.
3. Quantify business impact: (a) Simulate inventory holding cost using forecast vs. actual (holding cost per unit x excess inventory from over-forecast); (b) Assess stockout risk (lost sales from under-forecast x contribution margin per unit); (c) Calculate service level attainment (% of demand periods where inventory covered forecast demand).
4. Run statistical validation tests: (a) Diebold-Mariano test for pairwise model comparison (is Model A significantly better than Model B?); (b) Residual diagnostics: normality test (Jarque-Bera), autocorrelation test (Ljung-Box on residuals), heteroscedasticity test (Breusch-Pagan); (c) Prediction interval coverage: check that actual values fall within the predicted interval at the nominal confidence level (e.g., 95% of actuals within 95% PI).
5. Conduct stakeholder review: Present results to business stakeholders in a structured review meeting. Compare against current manual/existing forecasts. Document model limitations (edge cases where performance degrades) and known failure modes. Obtain formal sign-off before proceeding to deployment.
6. Model Evaluation Report (PDF) with all metrics, bias analysis, and business impact simulation; Diebold-Mariano test results; Stakeholder sign-off form (wet signature or digital approval in MLflow); and updated Model Card with evaluation results.

Output: Model Evaluation Report (PDF) with all metrics, bias analysis, and business impact simulation; Diebold-Mariano test results; Stakeholder sign-off form (wet signature or digital approval in MLflow); and updated Model Card with evaluation results.

SOP-06: Deployment & Integration

Objective: Deploy models to production with reliability, reproducibility, and zero-downtime rollback capability.

Procedure:

1. Package the model: (a) Classical ML models (LightGBM, Prophet) — serialize with joblib (preferred) or pickle, include feature names and preprocessing pipeline; (b) Deep learning models (CNN-LSTM) — export to TorchScript or ONNX format for language-agnostic serving; (c) Containerize the model and API code using Docker with a pinned base image and all dependencies version-locked.
2. Develop the serving API using FastAPI: implement POST endpoints for single and batch prediction; validate inputs with Pydantic models (type checking, range validation); add rate limiting (e.g., 100 requests/second per client); implement API key authentication; include health check endpoint (/health) for load balancer probing.
3. Integrate with downstream systems: (a) ERP connectors (SAP BAPI/RFC, Oracle REST API) for direct forecast injection into planning modules; (b) BI tool connectors (Power BI REST API, Tableau Web Data Connector) for dashboard data refresh; (c) Streaming platforms (Kafka producer for real-time forecast publishing to event-driven architectures).
4. Execute deployment strategy: (a) Canary deployment: route 5% traffic to new model -> monitor for 24 hours -> increase to 25% -> monitor 24 hours -> 100%; (b) Blue-green deployment: maintain two identical production environments, switch traffic instantly, keep previous version warm for 1 hour for instant rollback; (c) Maintain rollback runbook with step-by-step instructions for reverting to previous model version.
5. Perform post-deployment verification: (a) Smoke tests on all API endpoints with known inputs and expected outputs; (b) Latency validation: single prediction < 500ms (P99), batch of 1000 < 30 seconds; (c) Verify logging pipeline: all predictions logged with model version, input features, prediction, and timestamp; (d) Verify monitoring dashboards show live data.
6. Deployed API (Docker image in container registry), Integration Test Report (automated test suite passing), Production Runbook (deployment steps, rollback procedure, contact list), and monitoring dashboard URLs with alerts configured.

Output: Deployed API (Docker image in container registry), Integration Test Report (automated test suite passing), Production Runbook (deployment steps, rollback procedure, contact list), and monitoring dashboard URLs with alerts configured.

SOP-07: Monitoring & Continuous Improvement

Objective: Maintain model health in production and drive ongoing performance improvement through systematic monitoring.

Procedure:

1. Implement daily performance monitoring: (a) Forecast vs. actual tracking — compute MAPE, wMAPE, and bias for each forecast horizon (t+1, t+7, t+30) daily; (b) Prediction latency and throughput — track P50, P95, P99 latency and requests/second; (c) Error rate and availability — track HTTP 5xx rate and uptime % (target: 99.9%).
2. Implement weekly drift detection: (a) Data drift — compute PSI on all input features weekly; flag any feature with $PSI > 0.25$ for investigation by the data science team; (b) Concept drift — track rolling 30-day MAPE; if MAPE increases by $>20\%$ from the validation baseline, trigger investigation; (c) Target drift — compare actual demand distribution (mean, variance, quantiles) against training distribution using Kolmogorov-Smirnov test.
3. Configure retraining triggers: (a) Scheduled trigger: monthly retraining with the latest data (default cadence); (b) Performance trigger: rolling MAPE exceeds threshold (baseline + 20%); (c) Drift trigger: $PSI > 0.25$ on any top-10 feature OR concept drift detected via residual trend analysis; (d) Event trigger: major market change, new product launch, competitor entry, regulatory change.
4. Manage model lifecycle: (a) Archive models older than 2 retraining cycles from the active registry (move to archive with metadata preserved); (b) Maintain champion/challenger pipeline — always have at least one challenger model in shadow mode receiving production traffic; (c) Conduct quarterly model review meeting with stakeholders — present performance trends, drift analysis, and improvement proposals.
5. Operate the feedback loop: (a) Log all planner overrides (manual forecast adjustments) with reason codes; (b) Perform monthly root cause analysis on the top-10 largest forecast errors; (c) Maintain a prioritized improvement backlog with feature requests, bug fixes, and enhancement proposals.
6. Monthly Model Health Report (automated PDF generation from monitoring dashboards), Retraining Log (dates, triggers, data versions, metrics before/after), Quarterly Stakeholder Review presentation, and Prioritized Improvement Backlog in project management tool.

Output: Monthly Model Health Report (automated PDF generation from monitoring dashboards), Retraining Log (dates, triggers, data versions, metrics before/after), Quarterly Stakeholder Review presentation, and Prioritized Improvement Backlog in project management tool.

SOP-08: Governance & Compliance

Objective: Ensure responsible, auditable, and compliant ML operations throughout the model lifecycle.

Procedure:

1. Maintain model documentation (per model): (a) Model Card — purpose, intended use, training data summary, features used, performance metrics, limitations, ethical considerations, and out-of-scope use cases; (b) Training Run Log — date, data snapshot version, hyperparameters, metrics, and approver; (c) Deployment Log — deployment date, version deployed, approver, rollback plan reference.
2. Implement access control: (a) Role-Based Access Control (RBAC) for MLflow Model Registry — Data Scientists: read/write staging; ML Engineers: promote to production; Managers: approve transitions; (b) Production deployment requires two-person approval (4-eyes principle); (c) All prediction API access logged with client ID, timestamp, and request metadata.
3. Check for bias and fairness: (a) Analyze forecast error by region, product category, and customer segment quarterly; (b) If systematic bias > 10% MAPE difference between any two segments, initiate bias investigation; (c) Document any known biases and mitigations in the Model Card.
4. Ensure business continuity: (a) Maintain fallback model — a simpler, proven statistical model (e.g., seasonal exponential smoothing) that can be activated if the primary model fails; (b) Manual override capability — planners must be able to adjust forecasts in the BI dashboard with audit trail; (c) Disaster recovery plan — documented procedures for restoring ML infrastructure from backups, including model artifacts, feature store, and training data.
5. Conduct annual compliance audit: Review all active models against this SOP, update Model Cards, verify access controls, test disaster recovery procedures, and document findings in an Audit Report.
6. Model Cards for all production models, Audit Trail (MLflow + custom logging), Annual Compliance Audit Report, and Business Continuity Plan (tested annually).

Output: Model Cards for all production models, Audit Trail (MLflow + custom logging), Annual Compliance Audit Report, and Business Continuity Plan (tested annually).

SOP-09: Tools & Technology Standards

Objective: Define the standard technology stack for demand forecasting ML development to ensure consistency, maintainability, and team efficiency.

Procedure:

- 1. Adopt the following standard tools. Deviations require written justification and Engineering Lead approval.

Tool Standards

Capability	Standard Tool	Alternative (with approval)
Experiment Tracking	MLflow	Weights & Biases
Feature Store	Feast	Tecton
Data Versioning	DVC	Delta Lake
Pipeline Orchestration	Apache Airflow	Prefect, Dagster
Model Serving	FastAPI + Docker	BentoML, Seldon Core
Drift Monitoring	Evidently AI + Grafana	WhyLogs, NannyML
CI/CD	GitHub Actions	GitLab CI, Jenkins
Data Quality	Great Expectations	Soda, Deequ
Deep Learning Framework	PyTorch	TensorFlow / Keras
BI/Dashboard	Streamlit + Power BI	Tableau, Looker
Container Orchestration	Kubernetes	Docker Swarm (small deployments)
Secrets Management	HashiCorp Vault	AWS Secrets Manager, Azure Key Vault
Code Quality	Ruff + Black	pylint + flake8
Streaming Platform	Apache Kafka	Redpanda, RabbitMQ
Caching / Session Store	Redis	Memcached
Monitoring Dashboards	Grafana	Datadog, New Relic
Real-time Communication	WebSocket (FastAPI)	SSE (Server-Sent Events)
Local Orchestration	Docker Compose	Podman Compose

Output: Adopt the following standard tools. Deviations require written justification and Engineering Lead approval.

SOP-10: Streaming Data Ingestion & Real-Time Processing

Objective: Define procedures for ingesting, validating, and processing real-time demand data streams.

Procedure:

1. Configure Kafka topics per data source (sales.events, inventory.updates, external.weather, external.promotions) with appropriate partition counts based on throughput.
2. Implement schema validation using Avro/JSON Schema in the Kafka producer. Reject malformed messages to a dead-letter queue (DLQ) topic.
3. Deploy the aiokafka consumer as a FastAPI lifespan background task with batch upserts (100 records or 5-second flush) to the actuals table.
4. Configure idempotent producers (enable_idempotence=True) and manual commits (enable_auto_commit=False) for exactly-once semantics.
5. Monitor consumer lag using Kafka's built-in metrics. Alert when lag exceeds 1000 messages or 60 seconds.
6. Test the full stream end-to-end: produce test events, verify they appear in the actuals table within 10 seconds.
7. Streaming topology document, topic registry, consumer lag monitoring dashboard.

Output: Streaming topology document, topic registry, consumer lag monitoring dashboard.

SOP-11: Production Monitoring & Observability

Objective: Maintain full observability of ML models and infrastructure in production.

Procedure:

1. Deploy Grafana dashboards (demand-overview, forecast-accuracy, model-health) via docker-compose provisioning. Verify all panels render with live data.
2. Configure Evidently AI drift checks via APScheduler (daily at 06:00). Store results in drift_metrics table and auto-create alerts for detected drift.
3. Set up WebSocket heartbeat (30-second interval) for real-time dashboard connections. Monitor active connection count.
4. Enable Redis caching for dashboard API endpoints with configurable TTL. Monitor cache hit rate — target >80%.
5. Configure multi-tenant isolation: verify tenant_id is propagated through all queries. Test cross-tenant data isolation by querying as different tenants.
6. Review monitoring dashboards weekly: check for accuracy degradation, drift patterns, and resource utilization trends.
7. Operational runbook, monitoring dashboard URLs, alert escalation matrix.

Output: Operational runbook, monitoring dashboard URLs, alert escalation matrix.

Appendix A: Forecast Accuracy Metrics Quick Reference

Metric	Formula	Range	Best For
MAE	$\text{mean}(\text{actual} - \text{forecast})$	$[0, \infty)$	General comparison, same units as data
RMSE	$\text{sqrt}(\text{mean}((\text{actual} - \text{forecast})^2))$	$[0, \infty)$	Penalizing large errors heavily
MAPE	$\text{mean}(\text{actual} - \text{forecast} / \text{actual}) \times 100$	$[0, \infty)\%$	Percentage interpretability
sMAPE	$\text{mean}(2 \text{actual} - \text{forecast} / (\text{actual} + \text{forecast})) \times 100$	$[0, 200]\%$	Symmetric, bounded
wMAPE	$\text{sum}(\text{actual} - \text{forecast}) / \text{sum}(\text{actual})$	$[0, \infty)$	Revenue/cost-weighted accuracy (PRIMARY)
MASE	$\text{MAE}_{\text{model}} / \text{MAE}_{\text{naive}}$	$[0, \infty)$	Scale-independent, compare to naive
Pinball Loss	$\text{mean}(\max(q(y - \hat{y}), (q-1)(y - \hat{y})))$	$[0, \infty)$	Quantile/probabilistic forecasts
MPE	$\text{mean}((\text{actual} - \text{forecast}) / \text{actual}) \times 100$	$(-\infty, \infty)$	Bias direction detection

Table A.1: Standard forecast accuracy metrics

Metric Selection Guidelines:

- Use **wMAPE** as the primary metric when forecasts drive financial decisions (inventory, procurement) — it weights errors by volume/revenue impact.
- Use **MASE** when comparing models across different products with different demand scales.
- Use **Pinball Loss** when evaluating probabilistic forecasts (prediction intervals, quantile regression).
- Always report **MPE** alongside accuracy metrics to detect systematic bias.
- Never use **MAPE** when actual values can be zero or near-zero (intermittent demand) — use sMAPE or MASE instead.

Appendix B: CNN-BiLSTM Architecture Reference

```
Input: (batch_size, sequence_length, n_features)
|
Conv1D(filters=64, kernel_size=3) -> ReLU -> BatchNorm
|
Conv1D(filters=128, kernel_size=3) -> ReLU -> BatchNorm
|
MaxPooling1D(pool_size=2)
|
Dropout(0.3)
|
BiLSTM(units=128, return_sequences=True)
|
BiLSTM(units=64, return_sequences=False)
|
Dropout(0.3)
|
Dense(32, activation=ReLU)
|
Dense(n_outputs) <- Regression: linear | Classification: softmax
```

Training Configuration Reference:

Hyperparameter	Default Value	Search Range	Notes
Learning Rate	0.001	[1e-4, 1e-2] log-uniform	Adam optimizer
Batch Size	64	[16, 32, 64, 128, 256]	Scale based on GPU memory
Sequence Length	60 days	[30, 60, 90, 180, 365]	Match forecast horizon
Conv Filters (Layer 1)	64	[32, 64, 128]	First conv layer
Conv Filters (Layer 2)	128	[64, 128, 256]	Second conv layer
LSTM Units (Layer 1)	128	[64, 128, 256]	First BiLSTM layer
LSTM Units (Layer 2)	64	[32, 64, 128]	Second BiLSTM layer
Dropout Rate	0.3	[0.1, 0.2, 0.3, 0.4, 0.5]	After CNN and LSTM blocks
L2 Regularization	1e-4	[1e-5, 1e-3] log-uniform	On Dense layers
ReduceLROnPlateau Factor	0.5	Fixed	Patience: 10 epochs
EarlyStopping Patience	20	Fixed	Restore best weights: True